

Optimizing Reuse for Systems Engineering

Introducing Frame Technology as next step in
object-based development.

Table of Contents

- ☐ What is Frame Technology ?
- ☐ What are the pros & cons ?
- ☐ How should I approach FT ?
- ☐ Why should FT be considered at IRI ?
- ☐ How does FT Compliment Pattern Reuse ?
- ☐ Will FT allow us to leverage UML ?
- ☐ How does FT differ from OOD ?
- ☐ Will FT impact the way we approach OOD ?
- ☐ How can we get started ?
- ☐ Rethinking Staff Alignment ?
- ☐ List of References

What is Frame Technology ?

- ❑ Frame Technology (FT) is a method of applying Generative Programming techniques by using code selected from a library of code fragments, called frames.
- ❑ Generative Programming began evolving many decades and it focuses on using software to construct entire software applications.
- ❑ The automation of software production speeds up software development and it reduces the cost of development. Additionally, it improves the quality of software and reduces business exposure due to fewer errors. Finally, yet importantly, maintenance expenditure and the costs resulting from it will be also reduced.

What are the Pros & Cons ?

❑ Pros:

- FT results in 85% Reuse as a baseline.
- 70% Reduction in time-to-market.
- 80% Reduction in development costs.
- Potential to create separation from our competition.

❑ Cons:

- Need to align our OOD with FT (and Domain Engineering).
- Requires a paradigm shift in design & development.
- Requires an organizational shift to regard software as an asset.
- Requires a cultural shift away from date-driven deliverables to architecturally sound solutions.

How should we approach FT ?

- ❑ Currently, we develop in a single dimension (coding functional object per specification – UML or otherwise). FT introduces a method to construct software in multiple dimensions relying on relational constructs.
- ❑ Think about shifting your thinking to a new paradigm of development, similar to the shift we made in the early 80's to relational data stores. The difference is that the leap in productivity from applying relational techniques to the development process can be one to two orders of magnitude.
- ❑ FT is one method of “normalizing” the software “assets” to remove redundancy and introduce variants that can be easily changed within a relational class of objects called Frames.

Why should FT be considered ?

- ❑ First, we can introduce FT on top of current and future OOD. So, we can phase it in w/o disrupting ongoing development.
- ❑ Second, the technique allows us to iterate our Frames as we learn more about the business rules and continue to normalize our code. There is some rework, but once the Frames mature the significant reuse can be realized.
- ❑ Third, there is a tremendous potential to impact our ability to reduce project lifecycles, reduce product costs and move from a reactive to a proactive culture.

How does FT Complement Pattern Reuse ?

- ❑ The nature of the FT technique (and Generative Technology overall) is to identify re-occurring fragments of code and extract them from the “objects” and place them in the Frames. This code normalization process is a similar exercise that DBAs use to normalize their schemas. FT can emulate templates and inject variability to modify Patterns (as an example).

Will FT allow us to leverage UML ?

- ❑ UML can and should be used to model our business flows and the underlying objects. There are no adequate symbols to represent relational framing on top of the object framework at this time. However, the leading vendors are working to incorporate some of these capabilities as Generative Programming is taking hold in the development communities and is already recognized by OMG (Object Mgmt. Group) as the next logical step for object development.

How does FT differ from OOD ?

- ❑ Its not a question of difference as discussed earlier, FT complements OOD by taking it to its next logical level. Object design can proceed as normal and we can introduce Frames as we normalize the data.
- ❑ Obviously, its best to construct the components one time so we can reduce rework and begin leveraging these as true “assets” to be reused during the construction process. By introducing FT from the outset, we can begin building our asset library of reusable objects so we can begin leveraging and extending reuse throughout our new architecture.
- ❑ Disclaimer - Decisions on how to use components and FT and whatever else shall result from the Domain Analysis (which also results in Domain Model).

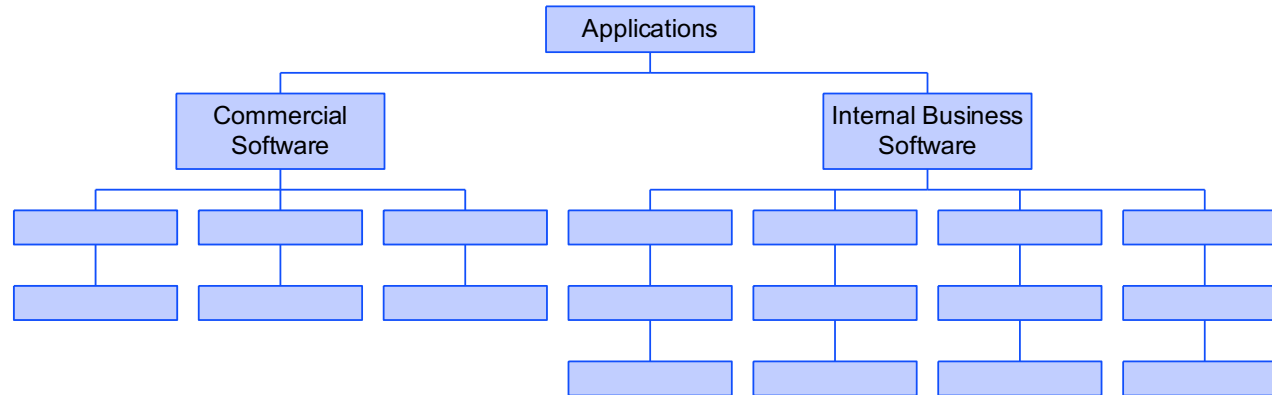
Will FT impact the way we approach OOD ?

- ❑ Yes and No. If we intend to introduce FT, there are certain guidelines for OOD that should be followed. However, we should follow these guidelines regardless to ensure we are aligned with the Domain Engineering model. Those guidelines are:
 - Minimize inheritance
 - Ensure OO structures follow the Domain Model
 - Utilize Control Classes to isolate process definitions, workflows & lower level controls
 - Create Object Classes that mirror the business elements as defined in the data architecture.
 - Generalization/Specialization and OSA/OBA techniques should be rigorously in this step. This is the essence of Domain Engineering: the continual analysis (to create reusable assets) and synthesis (to create applications using these assets)

How can we get started ?

- ❑ First, we need to embrace Domain Engineering (where Domain Analysis is a first step).
- ❑ Next, we need to start simple. Begin with a single project, where we can get a baseline for a project that is using OO (maybe IMT).
- ❑ Begin applying Frame Technology techniques against the application to redesign it and normalize it. While doing so we will most likely discover that Frame Technology is practical to be used (apart from OO).
- ❑ Ensure that OO structures follow the domain model for IRI, otherwise you could end up with poor frames. Make sure the pilot project follows the domain data model already defined by the Architecture Team.
- ❑ As you partition the elements around the Domain Entities, resist the temptation to create a taxonomy. Focus on decomposing the common elements into the "gene pool" for creating frames.
- ❑ Disclaimer: As with any architecture/design technology, poorly applied technique can result in an inferior application.

Rethinking the Staff Alignment



Most IT shops rely on a silo approach to deliver business functionality amongst teams of developers. Much of this effort is duplicated and promotes individual creativity, development and fragmented standards of development. We may want to begin planning a new alignment of resources around the assets-centric model rather than the project-centric one we currently embrace.

List of References:

❑ SMEs:

- Paul Basset – Internationally respected authority on software engineering. Co-founded Netron Inc., Co-author of the IEEE's Standard P1517: Software Reuse Lifecycle Processes.
- Stan Jarzabek – Associate Professor, Department of Computer Science, School of Computing National University of Singapore. Stan recently published XVCL's sources, objects, and documentation on <http://sourceforge.net/projects/fxvcl>.

❑ Vendors:

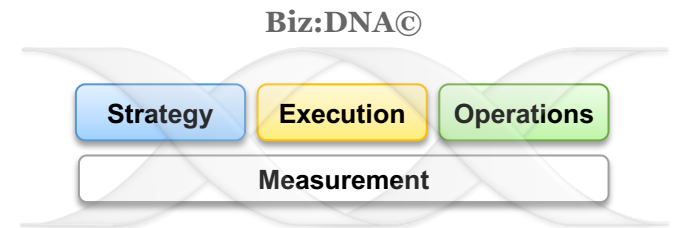
- Netron – Reverse Engineering tools and Generative Technology tools
- Delta Software Technology - Generative Technology tools

❑ Books & White Papers:

- Framing Software Reuse: Lessons From the Real World
- Generative Engineering

Thank You

Measure what matters
Create and deliver the right value
Build and nurture a winning culture
Proactively manage change & complexity



Phone: (847) 261-4332
Online: www.enablingvalue.com